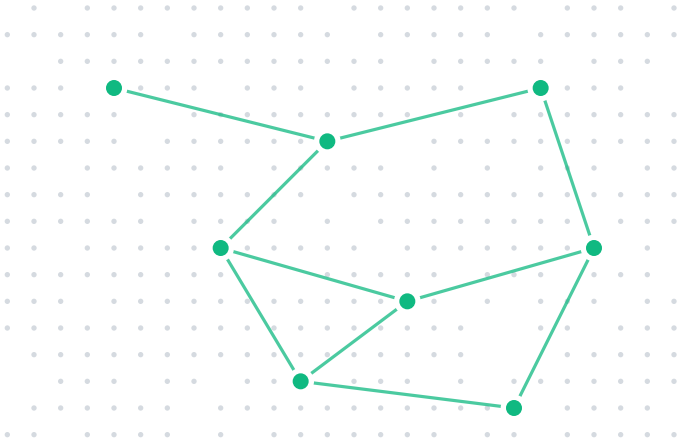


THE SYSTEM, END-TO-END

How Vectorless actually works.

Five components, three protocols, one retrieval primitive. This document walks the engine, the server, the SDKs, the MCP layer, and the managed control plane — with the flows that tie them together.

Audience — engineers, architects, platform teams.

PUBLISHED BY

Vectorless

Precision retrieval without embeddings

DOC

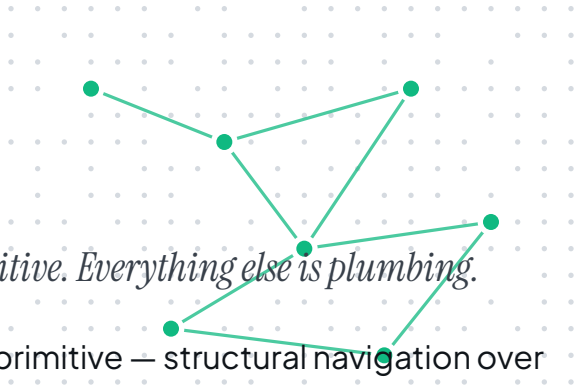
01

CONTENTS

Documentation map

PART / 00

00 System overview



Five components. One retrieval primitive. Everything else is plumbing.

Vectorless is a retrieval platform built around a single primitive — structural navigation over document trees. The five components below exist so that primitive can be consumed in whatever shape your stack prefers: as a library, an HTTP service, a typed client, an MCP tool, or a fully managed offering.

01	**vectorless-engine**	The retrieval primitive. Parses documents, builds trees, runs navigation.
02	**vectorless-server**	HTTP + SSE surface over the engine. REST-style, idempotent, queue-aware.
03	**vectorless-sdk**	Typed clients in TypeScript, Python, and Go. Thin wrappers over the REST API.
04	**vectorless-mcp**	Model Context Protocol bridge — exposes engine operations as MCP tools to LLM clients.
05	**vectorless-control-plane + dashboard**	Managed service — provisioning, multi-tenancy, quotas, usage, billing.

HOW THE PIECES DEPEND ON EACH OTHER

- `engine` is standalone — a Go library that also ships a CLI binary. It has no network, no auth.
- `server` embeds the engine and adds HTTP, auth, idempotency, async queueing.
- `sdks` speak to `server`, not the engine directly.
- `mcp` is a thin process that translates MCP tool calls into SDK calls.
- `control-plane` sits above N server instances, owns tenant state.

Deployment shapes

Every layer can be stopped at any boundary depending on how much of Vectorless you want to self-host.

SHAPE	YOU RUN	WE RUN	TYPICAL FIT
Library-only	engine (in-process)	—	Build-time use cases
Self-hosted	server + engine	—	Regulated, air-gapped
SDK-only	sdk in your app	server + control-plane	Most teams
MCP-enabled	mcp + your LLM client	server + control-plane	Agent builders
Fully managed	—	All five layers	Prototyping, pilots

ONE LINE TO ORIENT YOURSELF

The engine defines behaviour. The server defines the wire protocol. Everything else is a way to reach the server.

PART / 01

01 The engine

A Go library that turns documents into trees, and turns questions into paths through them.

`vectorless-engine` is the only component that knows how retrieval actually works. Everything else is a transport for it. The engine is a Go module (`github.com/hallelx2/vectorless-engine`, Go 1.25+) and ships a single statically-linked binary.

PACKAGE LAYOUT

`pkg/parser`	`format-specific`	PDF, HTML, Markdown, DOCX extractors
`pkg/tree`	`hierarchy model`	Node, summary, parent/child pointers, version hash
`pkg/ingest`	`pipeline`	parse 'tree 'summarise 'persist
`pkg/retrieval`	`navigation`	LLM-guided walk down the tree to a leaf
`pkg/storage`	`persistence`	on-disk tree store, append-only
`pkg/cache`	`memoisation`	per-query path cache
`pkg/queue`	`async`	ingest jobs, webhook callbacks
`pkg/db`	`metadata`	Postgres for docs/tenants (when server-embedded)
`pkg/config`	`yaml + env`	config.example.yaml is the authoritative schema

The retrieval pipeline

Five stages. Each stage is a function with pure-data input and pure-data output.

STAGE DETAILS

01 Ingest

Accepts bytes or a URL. Applies the format-specific parser in pkg/parser. Output is a raw structural tree — heading levels, numbered sections, table rows, figure anchors, all intact.

02 Parse

Converts raw structure to the canonical tree shape in pkg/tree. Each node has: id, kind, text, children[], parent, summary (empty at this stage), version_hash.

03 Summarise

For every non-leaf node, generates a one-paragraph summary of what lies under that subtree. LLM call; one-shot at ingest. Summaries are what the navigator reads, not the full text.

04 Navigate

Given a query, the navigator walks the tree top-down. At each node it asks the LLM: "which child answers this query?" with the candidates' summaries. The walk ends at a leaf or when the LLM says "stop, this node is the answer."

05 Return

The engine returns the final node, its full text, and the path — a structural address from root to leaf. Every step is logged to pkg/storage as an append-only audit record.

Core types (simplified)

PKG/TREE/TYPES.GO

```
type Node struct {
ID          string    // stable within a document version
Kind        NodeKind  // chapter | section | subsection | paragraph | table | figure
Text        string    // original text for leaves; title for internal nodes
Summary     string    // LLM-generated, used by navigator
Children    []*Node
Parent      *Node
VersionHash [32]byte  // sha-256 of parent doc version
}

type Path struct {
Segments []PathSegment // root 'leaf
}

type PathSegment struct {
NodeID string
Kind   NodeKind
Title  string
}
```

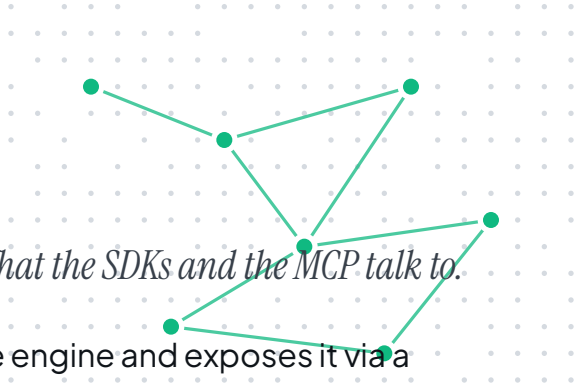
DETERMINISM CONTRACT

✓ NAVIGATION IS DETERMINISTIC PER (TREE, QUERY, MODEL)

The same tree version + same query + same navigation model always yields the same path. The engine pins the navigation model version per call and records it in the audit log. If the model is swapped, existing paths remain reproducible against their original model pin.

PART / 02

02 The server



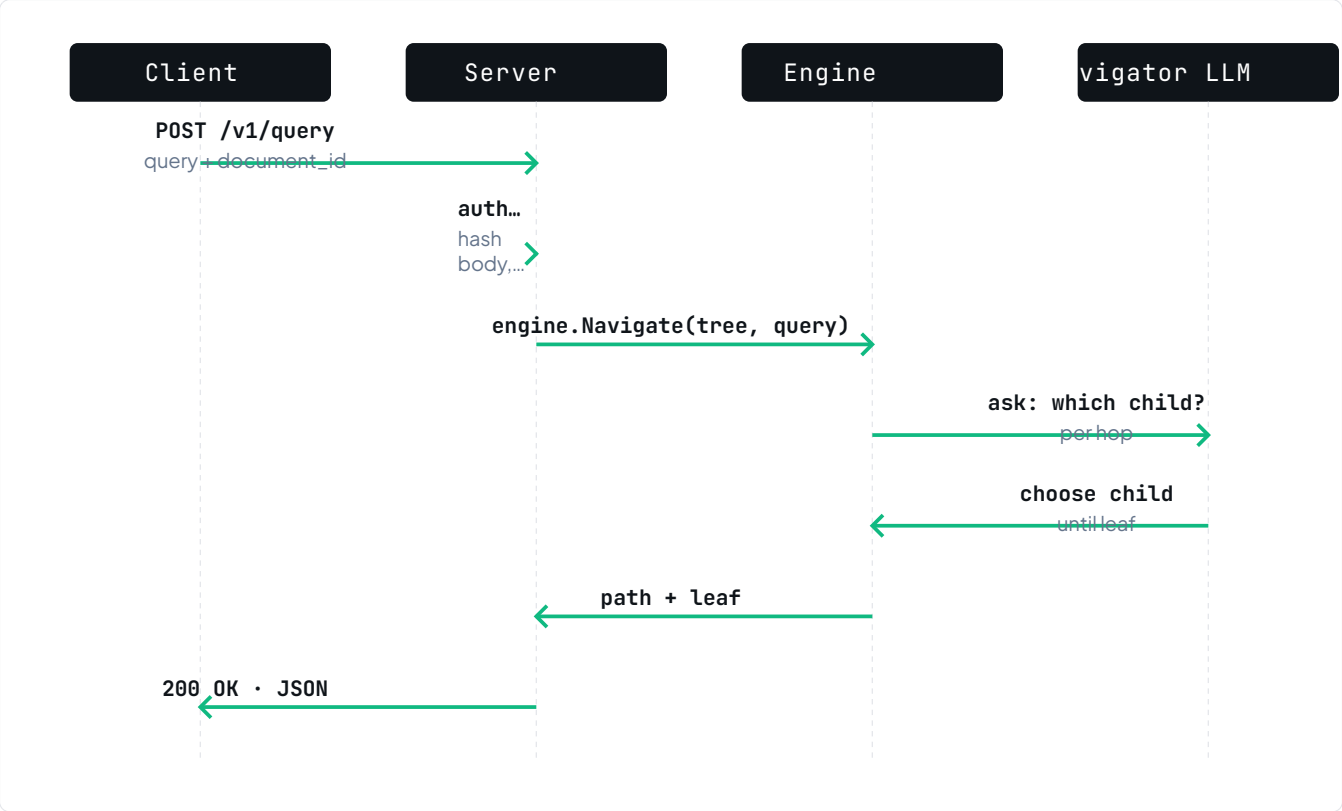
HTTP + SSE surface over the engine. What the SDKs and the MCP talk to.

`vectorless-server` is a Go service that embeds the engine and exposes it via a REST-style HTTP API and Server-Sent Events streams. Single binary, single config file, single process.

ENDPOINT SURFACE

GET	`/v1/health`	Liveness + build info.
GET	`/v1/documents`	List indexed documents for the tenant.
POST	`/v1/documents`	Submit a new document for ingest. Async-ready; idempotent by body hash.
GET	`/v1/documents/{id}/tree`	Fetch the structural tree for a document.
DELETE	`/v1/documents/{id}`	Remove a document + its tree + audit entries.
GET	`/v1/sections/{id}`	Fetch a single node by ID, including its ancestry path.
POST	`/v1/query`	Run a navigation query against one document. Returns path + leaf.
POST	`/v1/query/multi`	Run a query across N documents, returning one path per document.
POST	`/v1/query/stream`	Same as /v1/query but emits the navigation trail as SSE events.
POST	`/v1/query/multi/stream`	Same as /v1/query/multi as SSE events.

Request flow — POST /v1/query



REQUEST / RESPONSE SHAPES

POST /V1/QUERY — BODY

```
{
"query":       "What is the CET1 requirement for a G-SIB?",
"document_id": "doc_8a2f3e",
"options": {
"max_depth":     6,        // optional navigation depth limit
"include_trail": true      // return the full navigation trace
}
}
```

200 OK — RESPONSE


```
{
"path":   ["Basel III", "Pillar 1", "CET1 requirement", "G-SIB surcharge"],
"leaf": {
"id":     "node_b417ac",
"kind":   "paragraph",
"text":   "Globally systemically important banks ...",
"source": "basel_framework_cre42.pdf",
"version_hash": "3c9f...e8"
},
"trail": [
{ "step": 1, "candidates": 4, "chose": "Pillar 1", "elapsed_ms": 182 },
{ "step": 2, "candidates": 3, "chose": "CET1 requirement", "elapsed_ms": 141 },
{ "step": 3, "candidates": 2, "chose": "G-SIB surcharge", "elapsed_ms": 128 }
]
}
```

Streaming responses (SSE)

The `/stream` endpoints emit the navigator's decisions as they happen. Each `event` is one of three kinds.

TEXT/EVENT-STREAM · NAVIGATION EVENTS

```
event: step
data: {"step":1,"candidates":4,"chose":"Pillar 1","elapsed_ms":180}

event: step
data: {"step":2,"candidates":3,"chose":"CET1 requirement","elapsed_ms":141}

event: leaf
data: {"id":"node_b417ac","text":"...","path":["Basel III","Pillar 1",...]}

event: done
data: {"total_ms":451,"steps":3}
```

MIDDLEWARE STACK (APPLIED IN ORDER)

01	**CORS**	Allowed origins + GET/POST/DELETE/OPTIONS.
02	**Auth**	API key or JWT bearer; tenant extracted here.
03	**Rate limit**	Per-tenant token bucket; configurable.
04	**Idempotency**	Caches POST responses keyed by body hash.
05	**Queue router**	Long-running ingests get offloaded to QStash.
06	**Handler**	Engine call or tree fetch.

PART / 03

03 The SDKs

Typed, thin clients. They speak HTTP to the server — nothing more.

Three SDKs ship today — TypeScript, Python, Go — each mirroring the server's endpoint surface one-for-one. They exist to give your application code type-checked access to the API without hand-writing the HTTP calls.

LANGUAGE	PACKAGE	INSTALL
TypeScript	vectorless (npm)	npm install vectorless
Python	vectorless-sdk (PyPI)	pip install vectorless-sdk
Go	github.com/hallelx2/vectorless-sdk/go	go get github.com/hallelx2/vectorless-sdk/go

SHARED CLIENT CONTRACT

- Single `Client` constructor, takes a base URL and an API key.
- One method per endpoint — naming mirrors the REST path (`documents.list`, `query.run`, etc.).
- Streams surfaced as async iterators (TS/Python) or channels (Go).
- Retries with exponential backoff on 5xx; respected via `Retry-After` header on 429.
- Errors are typed — `AuthError`, `RateLimitError`, `ValidationError`, `ServerError`.

Usage examples

TYPESCRIPT

TS · QUERY + STREAM

```ts
import { Vectorless } from "vectorless";

const v = new Vectorless({
baseUrl: "https://api.vectorless.store",
apiKey: process.env.VECTORLESS_API_KEY!,
});

// One-shot query
const { path, leaf } = await v.query.run({
documentId: "doc_8a2f3e",
query: "CET1 requirement for a G-SIB",
});

// Stream the navigation trail
for await (const ev of v.query.stream({
documentId: "doc_8a2f3e",
query: "CET1 requirement for a G-SIB",
})) {
if (ev.type === "step") console.log(ev.chose, ev.elapsedMs);
if (ev.type === "leaf") console.log(ev.path.join(" > "));
}
```

PYTHON

PY · QUERY

```python
from vectorless import Vectorless

v = Vectorless(
base_url="https://api.vectorless.store",
api_key=os.environ["VECTORLESS_API_KEY"],
)

result = v.query.run(
document_id="doc_8a2f3e",
query="CET1 requirement for a G-SIB",
)
print(result.path)    # ['Basel III', 'Pillar 1', 'CET1 requirement', 'G-SIB surcharge']
print(result.leaf.text)
```

GO

GO · QUERY

```
import "github.com/hallelx2/vectorless-sdk/go/vectorless"

c := vectorless.New(
vectorless.WithBaseURL("https://api.vectorless.store"),
vectorless.WithAPIKey(os.Getenv("VECTORLESS_API_KEY")),
)

res, err := c.Query.Run(ctx, &vectorless.QueryRequest{
DocumentID: "doc_8a2f3e",
Query:      "CET1 requirement for a G-SIB",
})
```

PART / 04

04 The MCP layer

vectorless-mcp — expose the engine as Model Context Protocol tools.

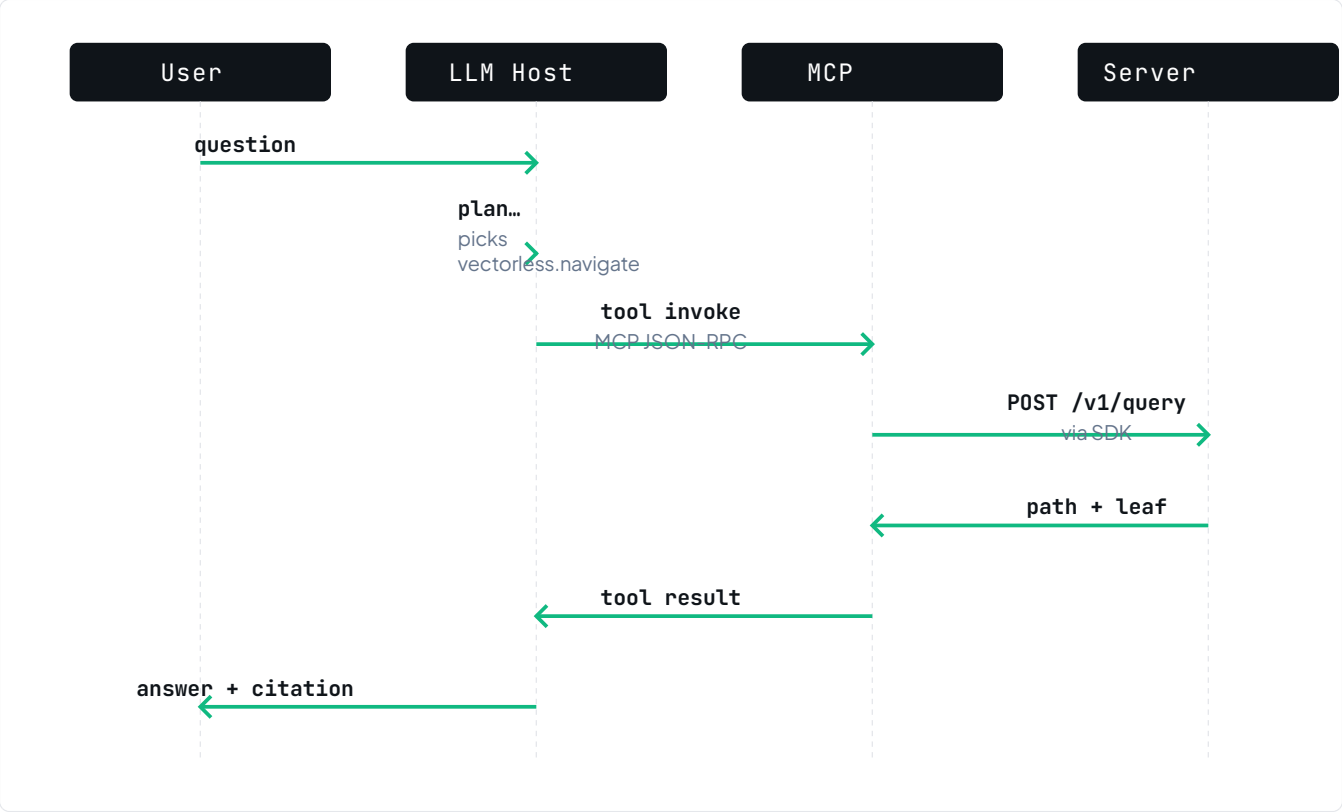
`vectorless-mcp` is a lightweight process that implements the *Model Context Protocol* server spec and translates MCP tool invocations into SDK calls. It runs as a local subprocess of an MCP-capable host (Claude Desktop, Cursor, Windsurf, etc.) or as a remote HTTP-based MCP endpoint.

TOOLS EXPOSED

`vectorless.list_documents`	`' Document[]`	Enumerate indexed documents for the authenticated tenant
`vectorless.get_tree`	`(doc_id) ' Tree`	Return the structural tree of a document
`vectorless.navigate`	`(doc_id, query) ' Path + Leaf`	Run a navigation query against a specific document
`vectorless.navigate_multi`	`(doc_ids[], query) ' Path[]`	Same as navigate, across multiple documents
`vectorless.get_section`	`(node_id) ' Section`	Fetch one node with its ancestry path
`vectorless.ingest_document`	`(url | bytes) ' job_id`	Submit a document for async ingest; poll or webhook

How an LLM host uses it

The host starts the MCP server, reads the tool manifest, and surfaces those tools to the language model. From the model's perspective, `vectorless.navigate` is just another callable. The flow below is what happens when a user asks a retrieval-backed question inside such a host.



TRANSPORT OPTIONS

MODE	TRANSPORT	USED BY
stdio	process stdin/stdout	Claude Desktop, Cursor, Windsurf — local-only
http	JSON-RPC over HTTP	Remote hosts, CI/CD agents
streamable	HTTP + SSE for long responses	Long-running navigate calls; streams the trail

✦ AUTH IN MCP MODE

> The MCP process carries a single API key for the tenant. Per-user auth is NOT handled at this layer — the MCP server presents itself as one principal to the Vectorless server. For multi-user separation, run one MCP instance per user.

PART / 05

05 The managed service

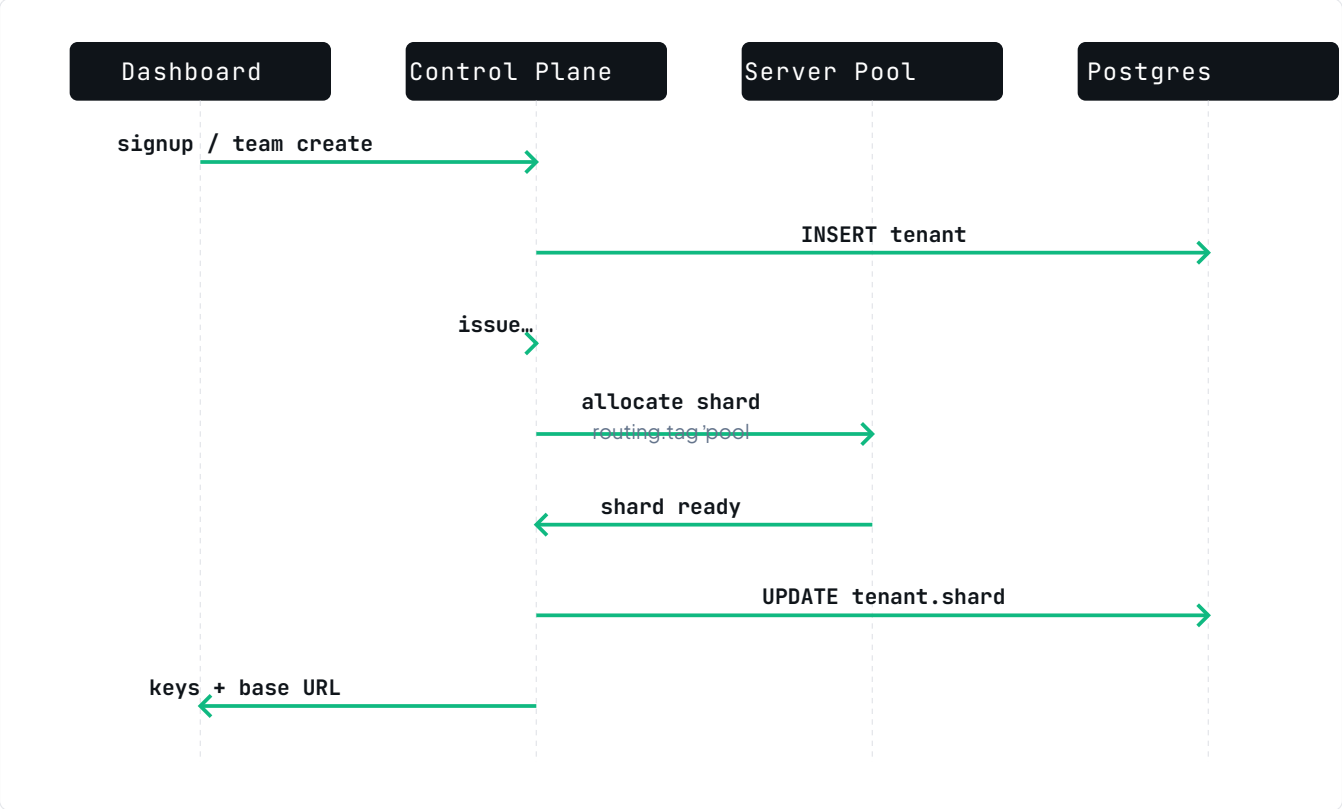
Control plane, dashboard, multi-tenancy — what you get when you don't want to run the server yourself.

The managed offering is two components — a Go `control-plane` and a Next.js `dashboard` — sitting above a pool of `server` instances. They exist to do the three things a single binary can't: tenant separation, provisioning, and usage/billing.

01	**dashboard (Next.js)**	User-facing console. Auth, document uploads, query history, billing. Next.js 15 + Turbo monorepo.
02	**control-plane (Go)**	Tenant registry, API key issuance, server-instance routing, usage accounting.
03	**server pool**	N instances of vectorless-server, one or more per tenant depending on plan.
04	**Postgres**	Tenants, documents metadata, API keys, audit trail.
05	**Object storage**	Raw document bytes + serialised trees.
06	**Queue (QStash)**	Async ingest jobs + webhook delivery.

Tenant provisioning flow

Every customer is a tenant. A tenant has: an ID, a plan, one or more API keys, a document quota, a query quota, and a routing tag that points at a server-pool shard.



ROUTING

Every request reaching the managed `api.vectorless.store` hits the control plane first. The control plane reads the API key, resolves it to a tenant + shard, and proxies the request to the appropriate `server` instance.

AUTHORIZATION HEADER FORMAT

```
Authorization: Bearer vl_live_sk_<32-char-random>

# resolves to:
#   tenant_id   = t_a8f3e2
#   plan        = growth
#   shard       = pool-us-east-1a
#   rate_limit  = 200 rpm, 12000 rpd
#   quotas      = 50000 docs, 5M queries/mo
```

Dashboard surfaces

- **Documents.** Upload UI, ingest status, per-document tree preview, delete.
- **Playground.** Ad-hoc query against any document; shows the navigation trail in real time (via the /stream endpoint).
- **API keys.** Issue, rotate, revoke. Scope keys to read-only or full-access.
- **Usage.** Queries/day, ingest bytes/day, latency p50/p95/p99, error rates.
- **Audit log.** Every query with path + trail + who ran it. Exportable as NDJSON.
- **Billing.** Plan, next invoice, payment method. Stripe-backed.

MULTI-TENANCY MODEL

LAYER	ISOLATION	NOTES
Dashboard	Session + CSRF	Per-user sessions; team members scoped by role
Control plane	API key 'tenant	Every request resolved before proxy
Server pool	Process + DB namespace	Growth / Scale tenants get dedicated instances
Storage	Bucket prefix per tenant	No cross-tenant read path exists
Audit	Postgres row-level	Queries filtered by tenant_id

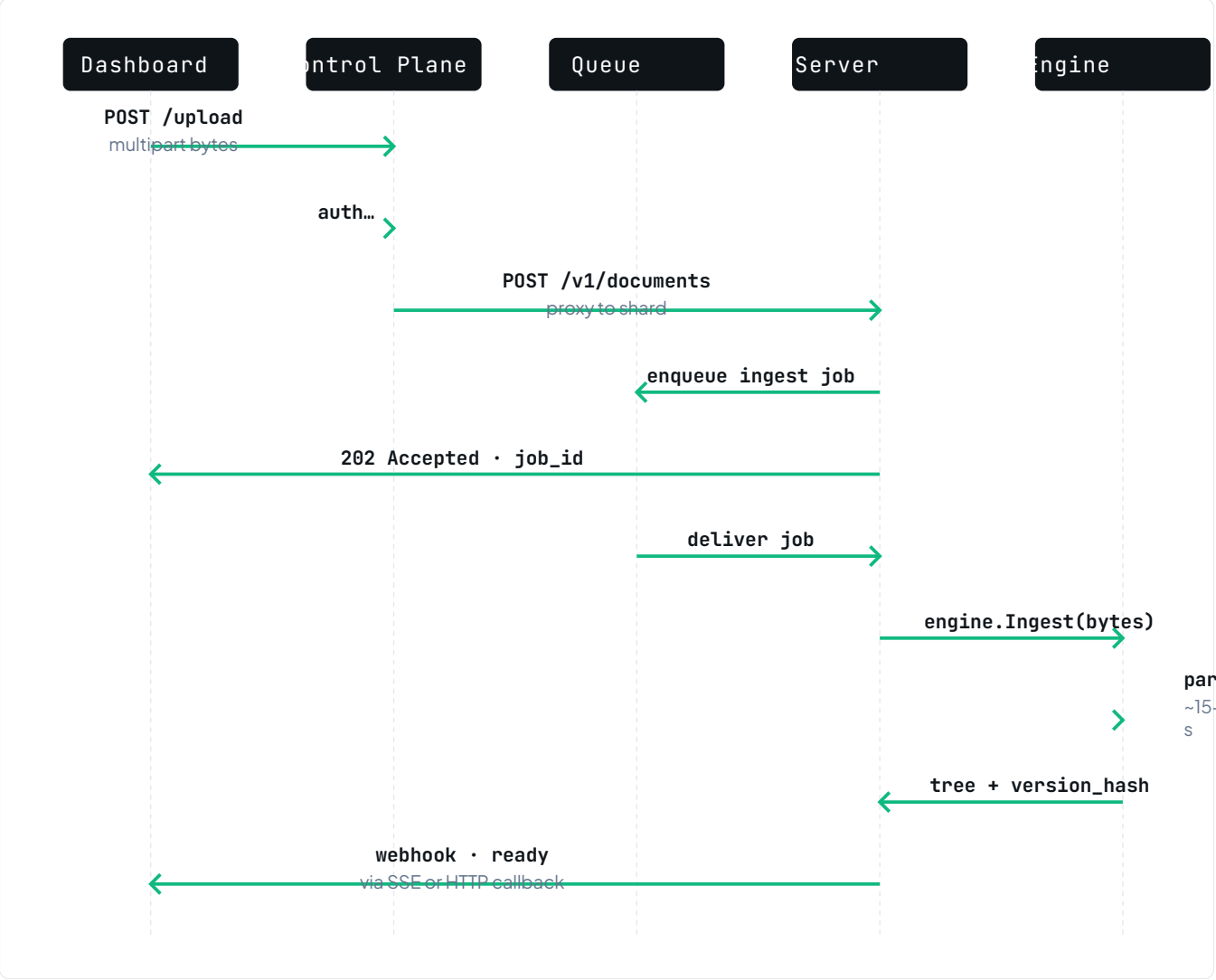
PART / 06

06 End-to-end flows

Two stories — one for ingest, one for query — showing every layer in play.

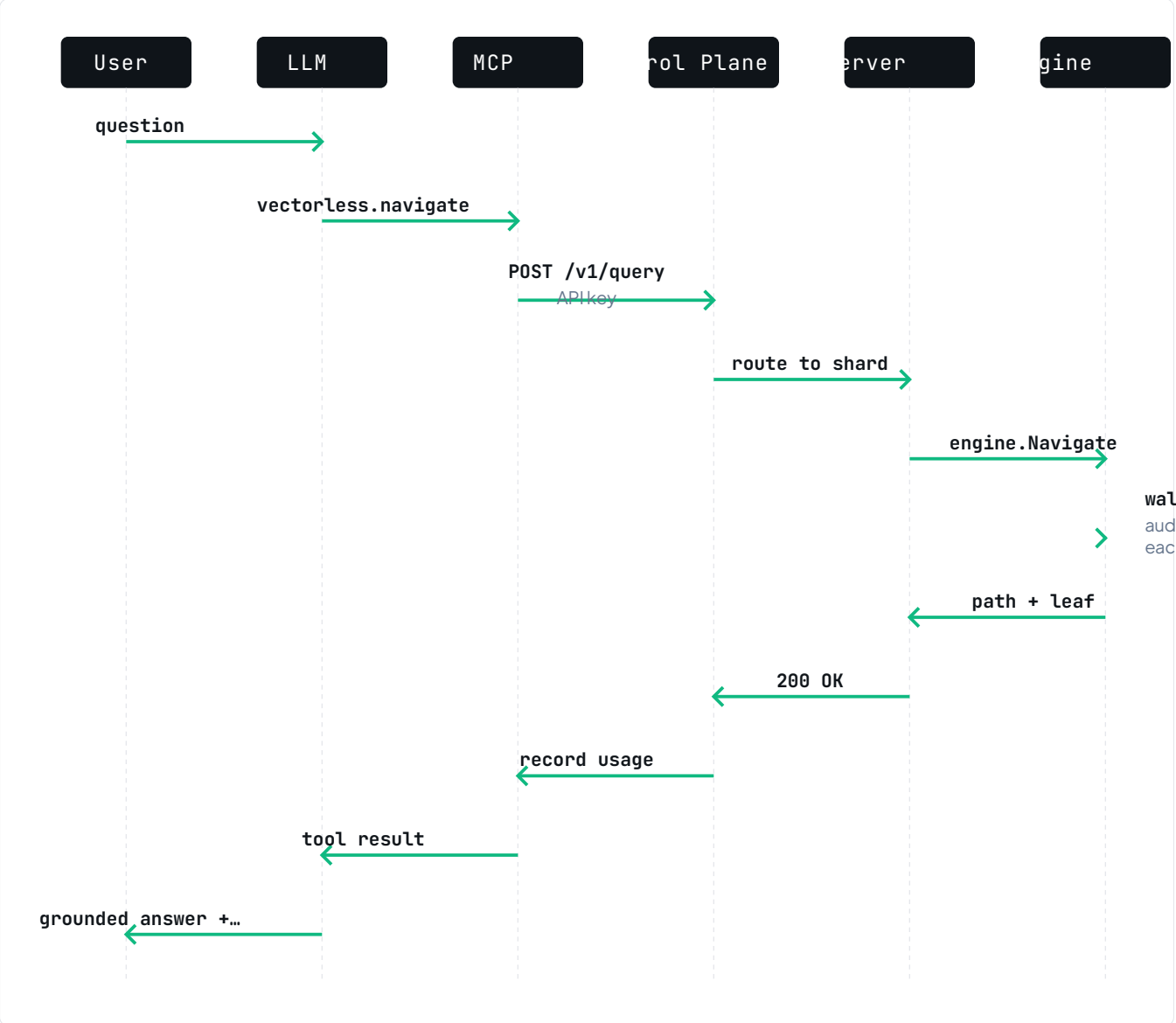
A · Document ingest

A user uploads a 280-page PDF through the dashboard. The managed service turns it into a tree, ready for queries.



B · Query from an agent

An agent running in a Claude Desktop session asks a clinical question. The query travels through every layer.



✓ THE INVARIANT ACROSS BOTH FLOWS

Every answer carries a version_hash and a path. Re-running the same query against the same version of the same document yields the identical path, always. That is the property that makes retrieval auditable — and it holds regardless of which layer the request enters at.

C · Configuration surface (summary)

All three Go binaries share a single configuration schema. The canonical reference is each project's `config.example.yaml`.

SELECTED KEYS

`server.bind`	`0.0.0.0:8080`	listen address
`server.cors.origins`	`[...]`	allowed origins
`engine.navigator.model`	`claude-opus-4-7`	LLM for navigation hops
`engine.navigator.max_depth`	`6`	hard cap on walk depth
`engine.summariser.model`	`claude-sonnet-4-6`	LLM for ingest-time summaries
`storage.kind`	`disk | s3 | gcs`	tree + raw-doc persistence
`db.url`	`postgres://...`	metadata
`queue.kind`	`qstash | inmem`	async ingest
`auth.keys.static`	`["sk_..."]`	dev; use control-plane in prod
`audit.retention_days`	`365`	navigation log ttl

CONFIG PRECEDENCE

For every key: explicit flag » env var (`VECTORLESS_*`) » yaml file » default. Secrets should always come from env, never yaml.